1.1.7. Bellman-Fo… 08:30

Bellman-Ford is a shortest path algorithm used to find the minimum distance from a single source vertex to all other vertices in a weighted directed graph. Unlike Dijkstra's algorithm, Bellman-Ford can handle negative edge weights and can also detect negative weight cycles.

Your task is to implement the **Bellman-Ford algorithm** and print the shortest distance and path from the given source vertex to every other vertex.

Input Format:

- The first line contains an integer V, the number of vertices in the graph.
- The second line contains an integer E, the number of edges in the graph.
- Next E lines: Each line contains three integers u , v , w - representing an edge from vertex u to vertex v with weight w.
- Last line contains an integer src, the source vertex from which shortest paths will be calculated.

Output Format:

- For each vertex v from 1 to V (except the source), the program prints:

If a node is reachable:
the program prints:

```
vertex shortest_distance path
```

Example:

```
4 6 1->2->4
```

Meaning:

Explorer

CTC79166.c

Submit

Debugger

```c
#include <stdio.h>
#include <stdlib.h>

#define INF 2e9

typedef struct {
    int u, v, w;
} Edge;

int V, E, source;
Edge edges[2005];
long long dist[1005];
int parent[1005];

void print_path(int v) {
    if (v == source) {
        printf("%d", source);
        return;
    }
    print_path(parent[v]);
    printf("->%d", v);
}

int main() {
    if (scanf("%d %d", &V, &E) != 2) return 0;

    for (int i = 0; i < E; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }
    scanf("%d", &source);

    for (int i = 1; i <= V; i++) {
```

‹ Prev Reset Submit Next ›

- Shortest distance from source = 6
- Path = 1 → 2 → 4

If a node is NOT reachable:
the program prints:

```
vertex INF None
```

Example:

```
5 INF None
```

Meaning:

- Vertex = 5
- Distance = INF (infinite)
- No path exists

If a negative cycle exists:
The program prints:

```
Negative cycle detected
```

Constraints:

- $1 <= V <= 1000$
- Use Bellman-Ford algorithm.

Note:

- Vertices are numbered from 1 to V.
- To run your custom test cases, strictly map your input and output layout with the visible test cases.

Sample Test Cases +

```
34          parent[i] = -1;
35      }
36      dist[source] = 0;
37
38  v   for (int i = 1; i
    <= V - 1; i++) {
39  v       for (int j = 0;
    j < E; j++) {
40              int u =
    edges[j].u;
41              int v =
    edges[j].v;
42              int w =
    edges[j].w;
43  v           if (dist[u]
    != INF && dist[u] + w <
    dist[v]) {
44                  dist[v]
    = dist[u] + w;
45
    parent[v] = u;
46              }
47          }
48      }
49
50  v   for (int j = 0; j <
    E; j++) {
51          int u =
    edges[j].u;
52          int v =
    edges[j].v;
53          int w =
    edges[j].w;
54  v       if (dist[u] !=
    INF && dist[u] + w <
    dist[v]) {
55
    printf("Negative cycle
    detected\n");
56              return 0;
57          }
58      }
59
```

```
    for (int i = 1; i
61  <= V; i++) {
        if (i ==
62  v source) continue;
        if (dist[i] ==
63  INF) {
            printf("%d
64  v INF None\n", i);
65      } else {
            printf("%d
66  %lld ", i, dist[i]);

67  print_path(i);

68  printf("\n");
69      }
70  }
71
72  return 0;
73  }
74
```